



27

Conteneurisation (Dockerfile)

US3.3 · C6

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Expliquer images, couches, cache
- › Écrire un Dockerfile multi-stage (build → runtime mince)
- › Durcir : non-root, .dockerignore, HEALTHCHECK
- › Prouver la prédiction en conteneur (Variante A : modèle livré)

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE docker run → /health 200 → /ready 200 → /predict-tabular 200 · non-root

1

Module 27 — les concepts clés

Théorie

■ Scénario atelier & enjeux

« Ça marche chez moi » — mais pas sur le serveur de l'usine. Le conteneur embarque l'application ET son environnement : ça tourne à l'identique partout.

Angle éthique / risque (C2) Une image qui s'exécute en root, c'est une faille grande ouverte. On tourne non-root pour réduire la surface d'attaque.

- Éco-conception : une image légère (multi-stage) = moins de stockage, des déploiements plus rapides et moins d'énergie.

■ Contexte & outils — pourquoi ce module

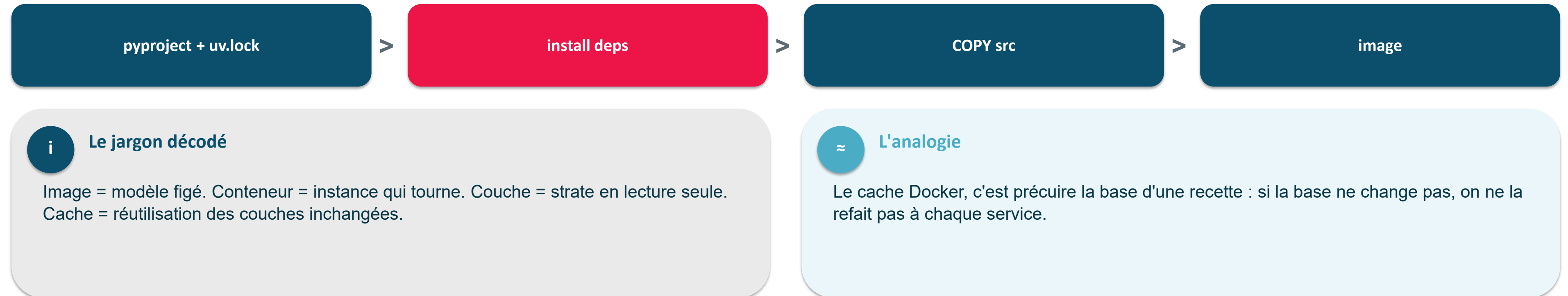
Contexte : « ça marche sur ma machine » ne suffit pas. Pourquoi : on fige TOUT l'environnement dans une image déployable à l'identique.

- Docker : empaquette code + dépendances + OS en une image
- Dockerfile : la recette (couches, cache, build multi-stage)
- Image slim + non-root : plus légère et plus sûre
- .dockerignore : exclut le superflu du contexte de build
- HEALTHCHECK : l'image sait dire si elle va bien

Objectif du module Une image légère, non-root, qui démarre l'API partout pareil.

■ Une image = des couches (et un cache)

Une image Docker est un empilement de couches mises en cache ; l'ordre des instructions décide de la vitesse de build.



EN UNE PHRASE Règle d'or : dépendances d'abord (cache stable), code en dernier.

■ Multi-stage : builder gros, livrer mince

On construit dans une image lourde (avec les outils) et on livre une image mince (juste le nécessaire).



? Le saviez-vous ?

Lancer un conteneur en root est un défaut classique : si l'app est compromise, l'attaquant est root DANS le conteneur. USER appuser réduit fortement l'impact.

→ Cas réel InduSense (Variante A)

Le modèle rf.joblib est livré dans l'image → /ready répond 200 dès docker run, sans entraînement.

EN UNE PHRASE Image runtime mince + non-root + HEALTHCHECK = sûre et rapide.

■ Le piège du modèle « disparu »

artifacts/ est gitignoré : un git clone n'embarque pas le modèle — il faut une exception explicite.

- Clone propre = pas de rf.joblib ; si .dockerignore l'exclut aussi → image sans modèle → /ready 503
- Parade (Variante A) : garder !artifacts/models/** ET COPY artifacts/models dans le runtime

! Attention — piège

« gitignoré » n'est pas « absent du build » : le contexte Docker et Git sont DEUX périmètres distincts.

+ Astuce de pro

uvicorn dans un conteneur DOIT écouter --host 0.0.0.0, sinon l'API est injoignable depuis l'hôte.

EN UNE PHRASE Variante A : le modèle voyage dans l'image, pas dans Git.

■ Le Dockerfile, en vrai (multi-stage, non-root)

On construit avec uv, on livre une image mince non-root, avec le modèle dedans (Variante A).

```
FROM python:3.13-slim AS build
COPY --from=ghcr.io/astral-sh/uv:0.11.19 /uv /usr/local/bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-install-project --no-dev
COPY src ./src
RUN uv sync --frozen --no-dev --no-editable
FROM python:3.13-slim AS runtime
RUN useradd -m -u 10001 appuser
WORKDIR /app
ENV PATH="/app/.venv/bin:$PATH" INDUSENSE_MODEL_DIR=/app/artifacts/models
COPY --from=build /app/.venv /app/.venv
COPY artifacts/models ./artifacts/models
USER appuser
HEALTHCHECK CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "indusense.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```



Cas réel InduSense

Variante A : modèle livré -> /ready 200 d'emblée. .dockerignore garde !artifacts/models.



Attention — piège

Sans --host 0.0.0.0, l'API est injoignable depuis l'hôte ; sans USER appuser, conteneur en root.

EN UNE PHRASE Build reproductible + image mince + non-root = la preuve C6 (implémenter).

■ À retenir en 3 points

- Image = recette ; conteneur = plat servi.
- On tourne non-root, avec un HEALTHCHECK.
- Le multi-stage donne une image légère.

2

Module 27 — TP guidé, correction au fil de l'eau

Travaux dirigés

TP — Dockerfile multi-stage

```
FROM python:3.13-slim AS build
COPY --from=ghcr.io/astral-sh/uv:0.11.19 /uv /usr/local/bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-install-project --no-dev
COPY src ./src
RUN uv sync --frozen --no-dev --no-editable
```

```
FROM python:3.13-slim AS runtime
RUN useradd -m -u 10001 appuser
WORKDIR /app
ENV PATH="/app/.venv/bin:$PATH" INDUSENSE_MODEL_DIR=/app/artifacts/models
COPY --from=build /app/.venv /app/.venv
COPY artifacts/models ./artifacts/models
USER appuser
HEALTHCHECK CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "indusense.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```


■ TP — build, run & preuve

```
docker build -t indusense:0.1.0 .
docker run -d --name indusense -p 8000:8000 -e INDUSENSE_API_KEY=dev-key indusense:0.1.0
curl.exe http://localhost:8000/health
curl.exe http://localhost:8000/ready
curl.exe -H "X-API-Key: dev-key" -H "Content-Type: application/json" --data-binary "@payload.json" http://localhost:8000/predict-tabular
docker exec indusense whoami
```

✓ **Preuve attendue** /health 200 → /ready 200 → /predict-tabular 200 ; docker exec whoami → appuser.

3

Module 27 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — alléger & scanner

Avancez à votre rythme. Le formateur reste disponible.

- Mesurer la taille (docker images) et la réduire
- OU scanner l'image avec Trivy (noter les HIGH/CRITICAL)

+ Posture en distanciel

Pas de Docker en local ? Le contrat de l'API reste prouvable via TestClient ; le docker run réel se scelle sur le poste formateur.

■ Definition of Done & transition

- DoD : image build+run, non-root, preuve /predict-tabular

Transition → Module 28 Une image ne suffit pas : on l'orchestre avec une base (docker-compose).